- Think of a smart contract as being a rules engine. It can serve as a gatekeeper controlling access to specific actions. If a nonmember airline requests an ASK action, the request will be reverted by the smart contract. This feature can be used by an off-chain application to prevent unauthorized users from completing Txs.
- As far as possible, design your smart contracts in such a way that most of the computations are performed off-chain. In the ASK Dapp, for example, the functions `ASKRequest()` and `ASKResponse()` simply record that the respective transactions have happened by transferring the parameter values into state variables. As another example, the decision to call the `settlePayment()` function will be made off-chain, based on the value of `success` in the `ASKResponse()` function.
- You must be aware of the immutable nature of smart contracts (and on-chain data) when you design blockchain-based systems. Think of smart contracts as being long-running programs and provide for repeated execution by using state properly and not using loops.
- Do not define a traditional database in a smart contract. If you do, there will be multiple replicated databases, one copy in each node. Also, do not port a database from a centralized system to a smart contract. Any centralized database has to be off-chain.
- You can design systems with fiat currency payment systems for traditional operations and cryptocurrency payments for the blockchain-based operations. You used ether for the `settlePayment()` function in the ASK Dapp, but the original airline tickets were probably bought with fiat currency such as U.S. dollars or Guatemalan quetzal.